

Imperative Programming in OCaml

Programming Languages

Sujit Kumar Chakrabarti

IITB

Imperative Programming

Side-effects

```
x = 10  
x = x + 20
```

- `x` is defined in line 1.
- `x` is modified in line 2.

Imperative Programming

Side-effects

Undesirable

- Side-effects may be dangerous – cause most of software bugs
- Example: Global variables

Imperative Programming

Side-effects

Undesirable

- Side-effects may be dangerous – cause most of software bugs
- Example: Global variables

Unavoidable

- IO
- Impossible to avoid: e.g. Hash tables
- Inefficient to avoid: e.g. graphs

Imperative Programming

Imperative constructs in OCaml

- Reference cells and commands
- Arrays
- Mutable fields
- Loops: for and while loops

Imperative Programming

Imperative constructs in OCaml - Reference cells

■ Defining reference cells

```
# let x = ref 1;;  
val x : int ref = {contents = 1}
```

■ Reading the value

```
# !x;;  
- : int = 1
```

Similar
to
dereferencing
operator (*)
in C/C++.

Imperative Programming

Imperative constructs in OCaml - Reference cells

$(:=) \quad x \quad (!x + 1)$

■ Changing the value

```
# x := !x + 1;;  
- : unit = ()  
# !x;;  
- : int = 2
```

Imperative Programming

Imperative constructs in OCaml - Arrays

- Defining arrays

Arrays are mutable.

```
# let a = [| 1; 2; 3 |];;  
val a : int array = [|1; 2; 3|]
```

- Reading the value of an element

```
# a.(2);;  
- : int = 3
```

- Changing the value of an element

```
# a.(2) <- 300;;  
- : unit = ()  
# a;;  
- : int array = [|1; 2; 300|]
```

- Reference: <https://v2.ocaml.org/api/Array.html>

Imperative Programming

Imperative constructs in OCaml - Arrays

Activity

Implement a function `copy_array` that copies all elements of an array to another of equal or greater length.

Imperative Programming

Imperative constructs in OCaml - Mutable fields

- Defining a record type with mutable field

```
# type name = { fname : string; mutable surname : string; };;  
type name = { fname : string; mutable surname : string; }
```

Imperative Programming

Imperative constructs in OCaml - Mutable fields

- Defining a record type with mutable field

```
# type name = { fname : string; mutable surname : string; };;  
type name = { fname : string; mutable surname : string; }
```

- Instantiation

```
# let myname = { fname = "Sujit"; surname = "Chakrabarti" };;  
val myname : name = {fname = "Sujit"; surname = "Chakrabarti"}  
# myname;;  
- : name = {fname = "Sujit"; surname = "Chakrabarti"}
```

Imperative Programming

Imperative constructs in OCaml - Mutable fields

- Defining a record type with mutable field

```
# type name = { fname : string; mutable surname : string; };;  
type name = { fname : string; mutable surname : string; }
```

- Instantiation

```
# let myname = { fname = "Sujit"; surname = "Chakrabarti" };;  
val myname : name = {fname = "Sujit"; surname = "Chakrabarti"}  
# myname;;  
- : name = {fname = "Sujit"; surname = "Chakrabarti"}
```

- Reading the value of a field

```
# myname.fname;;  
- : string = "Sujit"
```

Imperative Programming

Imperative constructs in OCaml - Mutable fields

- Changing the value

```
# myname.surname <- "Banerjee";;  
- : unit = ()  
# myname;;  
- : name = {fname = "Sujit"; surname = "Banerjee"}
```

Imperative Programming

Imperative constructs in OCaml - While Loop

[illegible]

Imperative constructs in OCaml - Loops

For loop

[illegible]

Imperative Programming

Imperative constructs in OCaml - Loops

Activity

Modify the `copy_array` function using

- while loop
- for loop.

Imperative Programming

Discipline

- Easy to reduce OCaml to 'just another C'
- Not adviceable!
- OCaml design favours functional style:
 - All variables are immutable by default
- Use imperative style where it is unavoidable or would lead to performance issues
- Carefully encapsulate imperative parts
- Overall architecture should follow functional design

Reading Exercise

- Hash tables
- Memoisation